Lab 6

House price prediction using MLP

Task : Understand the Effect of the Following Two Points

- Accuracy Curve (Train, Test, and Validation)
- Effect of Changing Batch Size

In [ ]:

```
# ==========================================
# 1. Import Libraries
# ==========================================
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

# ==========================================
# 2. Load Dataset as DataFrame
# ==========================================
housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target

print("Dataset Shape:", df.shape)
print(df.head())

# ==========================================
# 3. Feature-Target Split
# ==========================================
X = df.drop("Price", axis=1)
y = df["Price"]

# ==========================================
# 4. Train-Test Split
# ==========================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ==========================================
# 5. Feature Scaling (VERY IMPORTANT)
# ==========================================
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```python
X_test = scaler.transform(X_test)

# ==========================================
# 6. Build Multilayer Model
# ==========================================
model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # Linear activation for regression
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

model.summary()

# ==========================================
# 7. Train Model
# ==========================================
history = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=32,
    verbose=1
)

# ==========================================
# 8. Evaluate Model
# ==========================================
loss, mae = model.evaluate(X_test, y_test)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nTest MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)

# ==========================================
# 9. Plot Training Curve
# ==========================================
plt.figure()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("MSE Loss")
plt.show()
```

```python
# ========================================
# 10. Actual vs Predicted Plot
# ========================================
plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()
```

```
Dataset Shape: (20640, 9)
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  Price
0    -122.23  4.526
1    -122.22  3.585
2    -122.24  3.521
3    -122.25  3.413
4    -122.25  3.422
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarni
ng: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequent
ial models, prefer using an `Input(shape)` object as the first layer in the model in
stead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	1,152
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 1.1478 - mae: 0.7297 - val_loss: 0.
4659 - val_mae: 0.4743
Epoch 2/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3962 - mae: 0.4413 - val_loss: 0.
3899 - val_mae: 0.4369
Epoch 3/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.3507 - mae: 0.4170 - val_loss: 0.
3727 - val_mae: 0.4256
Epoch 4/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.3287 - mae: 0.4070 - val_loss: 0.
3549 - val_mae: 0.4010
Epoch 5/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.3227 - mae: 0.3921 - val_loss: 0.
3366 - val_mae: 0.4005
Epoch 6/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.3122 - mae: 0.3818 - val_loss: 0.
3348 - val_mae: 0.4078
Epoch 7/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2967 - mae: 0.3778 - val_loss: 0.
3328 - val_mae: 0.4063
Epoch 8/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2905 - mae: 0.3717 - val_loss: 0.
3319 - val_mae: 0.4195
Epoch 9/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2857 - mae: 0.3657 - val_loss: 0.
3183 - val_mae: 0.3887
Epoch 10/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2767 - mae: 0.3613 - val_loss: 0.
3296 - val_mae: 0.4040
Epoch 11/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2791 - mae: 0.3622 - val_loss: 0.
3470 - val_mae: 0.4259
Epoch 12/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2722 - mae: 0.3570 - val_loss: 0.
2947 - val_mae: 0.3713
Epoch 13/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2653 - mae: 0.3511 - val_loss: 0.
3070 - val_mae: 0.3789
Epoch 14/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2698 - mae: 0.3550 - val_loss: 0.
3124 - val_mae: 0.3876
Epoch 15/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2587 - mae: 0.3469 - val_loss: 0.
3251 - val_mae: 0.3766
Epoch 16/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2560 - mae: 0.3471 - val_loss: 0.
3099 - val_mae: 0.3779
Epoch 17/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2571 - mae: 0.3460 - val_loss: 0.
2998 - val_mae: 0.3646
Epoch 18/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2564 - mae: 0.3472 - val_loss: 0.
2897 - val_mae: 0.3593
Epoch 19/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2567 - mae: 0.3450 - val_loss: 0.
```

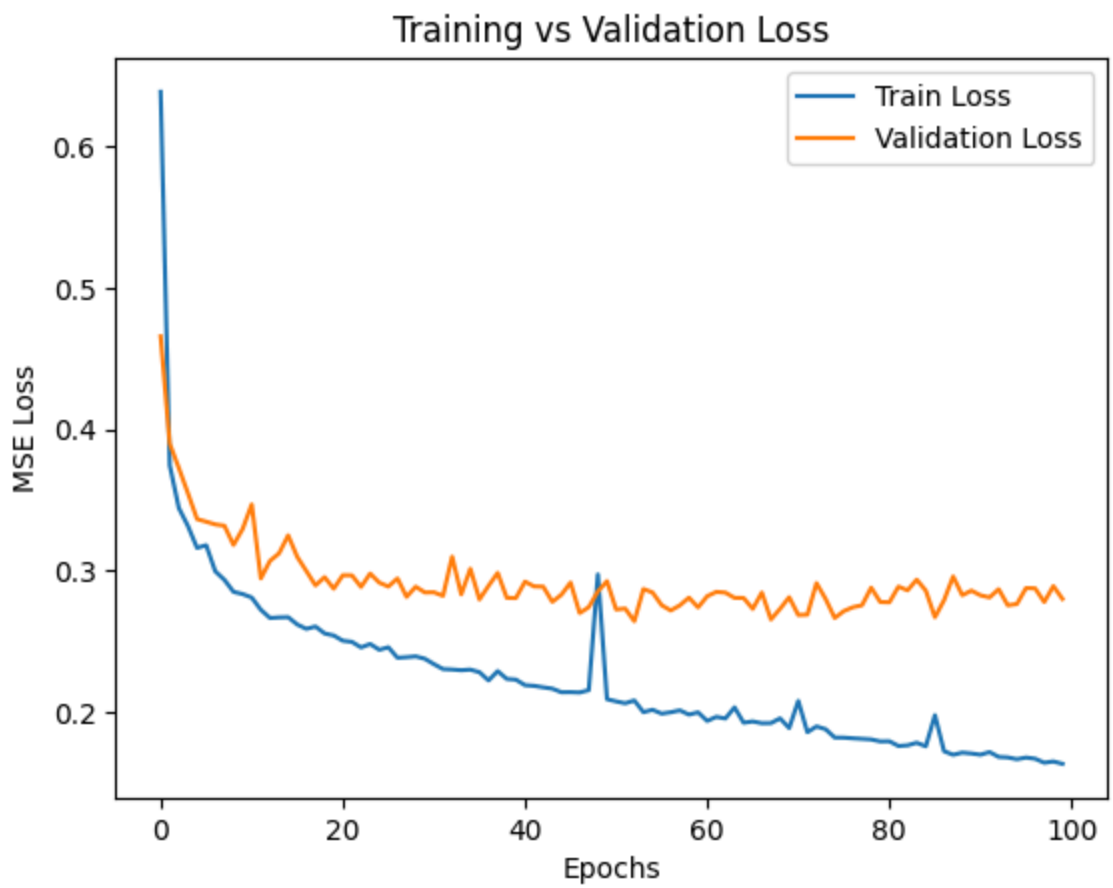
```
2956 - val_mae: 0.3648
Epoch 20/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2663 - mae: 0.3505 - val_loss: 0.
2874 - val_mae: 0.3555
Epoch 21/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2489 - mae: 0.3391 - val_loss: 0.
2969 - val_mae: 0.3615
Epoch 22/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2493 - mae: 0.3410 - val_loss: 0.
2968 - val_mae: 0.3852
Epoch 23/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2344 - mae: 0.3330 - val_loss: 0.
2886 - val_mae: 0.3556
Epoch 24/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2525 - mae: 0.3426 - val_loss: 0.
2981 - val_mae: 0.3706
Epoch 25/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2426 - mae: 0.3386 - val_loss: 0.
2917 - val_mae: 0.3761
Epoch 26/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2486 - mae: 0.3394 - val_loss: 0.
2889 - val_mae: 0.3669
Epoch 27/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.2357 - mae: 0.3324 - val_loss: 0.
2946 - val_mae: 0.3773
Epoch 28/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2409 - mae: 0.3356 - val_loss: 0.
2816 - val_mae: 0.3581
Epoch 29/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2350 - mae: 0.3340 - val_loss: 0.
2888 - val_mae: 0.3609
Epoch 30/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2395 - mae: 0.3319 - val_loss: 0.
2847 - val_mae: 0.3585
Epoch 31/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2337 - mae: 0.3298 - val_loss: 0.
2849 - val_mae: 0.3677
Epoch 32/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2244 - mae: 0.3226 - val_loss: 0.
2823 - val_mae: 0.3515
Epoch 33/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2287 - mae: 0.3280 - val_loss: 0.
3101 - val_mae: 0.3699
Epoch 34/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2243 - mae: 0.3256 - val_loss: 0.
2834 - val_mae: 0.3567
Epoch 35/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2347 - mae: 0.3281 - val_loss: 0.
3015 - val_mae: 0.3662
Epoch 36/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2231 - mae: 0.3245 - val_loss: 0.
2798 - val_mae: 0.3510
Epoch 37/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2295 - mae: 0.3255 - val_loss: 0.
2894 - val_mae: 0.3589
Epoch 38/100
```

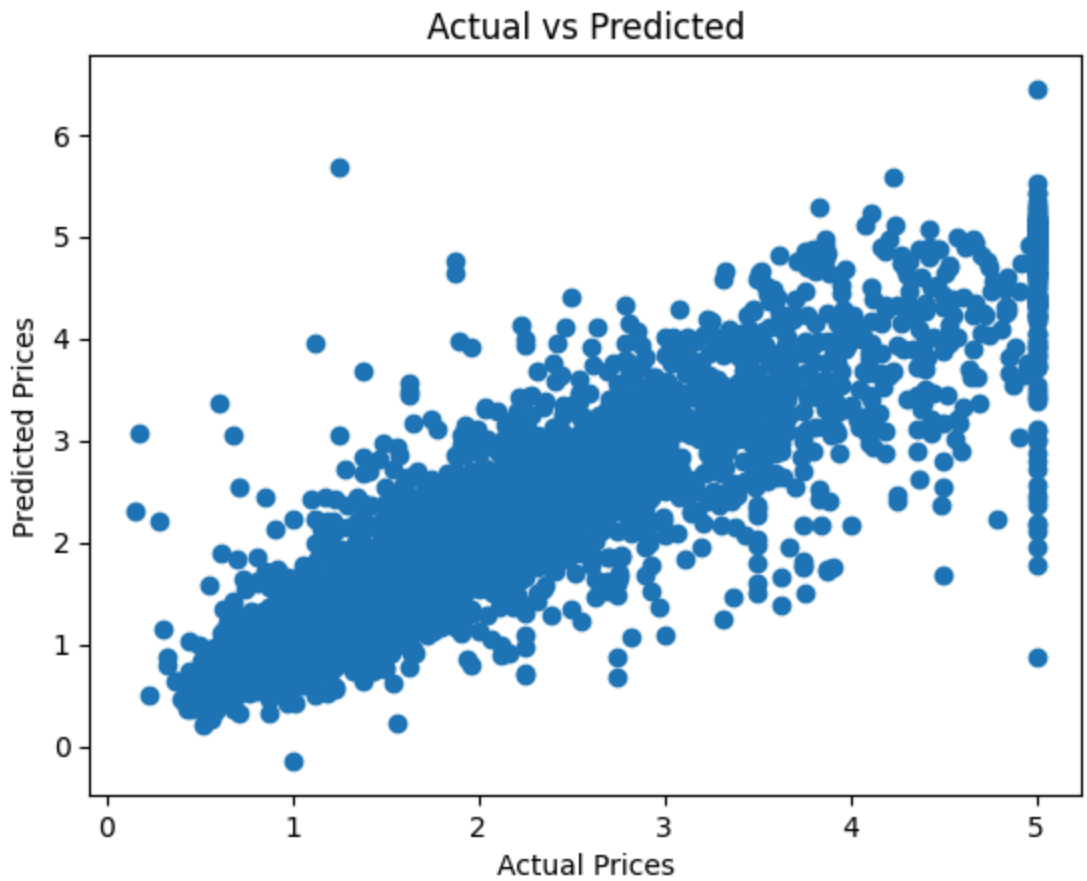
```
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2186 - mae: 0.3185 - val_loss: 0.
2986 - val_mae: 0.3595
Epoch 39/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2171 - mae: 0.3171 - val_loss: 0.
2809 - val_mae: 0.3510
Epoch 40/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2199 - mae: 0.3205 - val_loss: 0.
2808 - val_mae: 0.3523
Epoch 41/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2124 - mae: 0.3153 - val_loss: 0.
2924 - val_mae: 0.3539
Epoch 42/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2187 - mae: 0.3187 - val_loss: 0.
2890 - val_mae: 0.3673
Epoch 43/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2078 - mae: 0.3134 - val_loss: 0.
2888 - val_mae: 0.3569
Epoch 44/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2145 - mae: 0.3166 - val_loss: 0.
2781 - val_mae: 0.3499
Epoch 45/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2087 - mae: 0.3132 - val_loss: 0.
2831 - val_mae: 0.3622
Epoch 46/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2155 - mae: 0.3183 - val_loss: 0.
2917 - val_mae: 0.3555
Epoch 47/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2156 - mae: 0.3176 - val_loss: 0.
2702 - val_mae: 0.3475
Epoch 48/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2131 - mae: 0.3150 - val_loss: 0.
2743 - val_mae: 0.3442
Epoch 49/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.2217 - mae: 0.3088 - val_loss: 0.
2854 - val_mae: 0.3514
Epoch 50/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.2097 - mae: 0.3127 - val_loss: 0.
2927 - val_mae: 0.3596
Epoch 51/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2097 - mae: 0.3135 - val_loss: 0.
2727 - val_mae: 0.3433
Epoch 52/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1995 - mae: 0.3053 - val_loss: 0.
2733 - val_mae: 0.3492
Epoch 53/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2054 - mae: 0.3121 - val_loss: 0.
2644 - val_mae: 0.3487
Epoch 54/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1976 - mae: 0.3062 - val_loss: 0.
2872 - val_mae: 0.3509
Epoch 55/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1914 - mae: 0.3006 - val_loss: 0.
2847 - val_mae: 0.3533
Epoch 56/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1950 - mae: 0.3034 - val_loss: 0.
2757 - val_mae: 0.3518
```

```
Epoch 57/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1980 - mae: 0.3024 - val_loss: 0.
2720 - val_mae: 0.3534
Epoch 58/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2072 - mae: 0.3101 - val_loss: 0.
2756 - val_mae: 0.3578
Epoch 59/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1886 - mae: 0.3006 - val_loss: 0.
2811 - val_mae: 0.3566
Epoch 60/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1903 - mae: 0.3006 - val_loss: 0.
2741 - val_mae: 0.3500
Epoch 61/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1876 - mae: 0.2986 - val_loss: 0.
2821 - val_mae: 0.3561
Epoch 62/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1946 - mae: 0.3047 - val_loss: 0.
2851 - val_mae: 0.3703
Epoch 63/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1929 - mae: 0.3032 - val_loss: 0.
2846 - val_mae: 0.3510
Epoch 64/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.2072 - mae: 0.3080 - val_loss: 0.
2809 - val_mae: 0.3717
Epoch 65/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1926 - mae: 0.3039 - val_loss: 0.
2808 - val_mae: 0.3629
Epoch 66/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1909 - mae: 0.3014 - val_loss: 0.
2731 - val_mae: 0.3415
Epoch 67/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1877 - mae: 0.2981 - val_loss: 0.
2846 - val_mae: 0.3463
Epoch 68/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1975 - mae: 0.3073 - val_loss: 0.
2656 - val_mae: 0.3456
Epoch 69/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1841 - mae: 0.2943 - val_loss: 0.
2731 - val_mae: 0.3526
Epoch 70/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1882 - mae: 0.2999 - val_loss: 0.
2813 - val_mae: 0.3471
Epoch 71/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1885 - mae: 0.2958 - val_loss: 0.
2688 - val_mae: 0.3459
Epoch 72/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1855 - mae: 0.2957 - val_loss: 0.
2690 - val_mae: 0.3500
Epoch 73/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1887 - mae: 0.2987 - val_loss: 0.
2913 - val_mae: 0.3800
Epoch 74/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1827 - mae: 0.2930 - val_loss: 0.
2801 - val_mae: 0.3553
Epoch 75/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1772 - mae: 0.2911 - val_loss: 0.
```

```
2667 - val_mae: 0.3487
Epoch 76/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1839 - mae: 0.2971 - val_loss: 0.
2715 - val_mae: 0.3447
Epoch 77/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1787 - mae: 0.2912 - val_loss: 0.
2743 - val_mae: 0.3524
Epoch 78/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1767 - mae: 0.2920 - val_loss: 0.
2757 - val_mae: 0.3504
Epoch 79/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1717 - mae: 0.2865 - val_loss: 0.
2881 - val_mae: 0.3465
Epoch 80/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1765 - mae: 0.2904 - val_loss: 0.
2780 - val_mae: 0.3545
Epoch 81/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1709 - mae: 0.2877 - val_loss: 0.
2778 - val_mae: 0.3509
Epoch 82/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1762 - mae: 0.2901 - val_loss: 0.
2889 - val_mae: 0.3657
Epoch 83/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1724 - mae: 0.2889 - val_loss: 0.
2862 - val_mae: 0.3569
Epoch 84/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1712 - mae: 0.2871 - val_loss: 0.
2938 - val_mae: 0.3593
Epoch 85/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1719 - mae: 0.2882 - val_loss: 0.
2860 - val_mae: 0.3580
Epoch 86/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - loss: 0.1895 - mae: 0.2878 - val_loss: 0.
2672 - val_mae: 0.3442
Epoch 87/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1672 - mae: 0.2825 - val_loss: 0.
2792 - val_mae: 0.3515
Epoch 88/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1714 - mae: 0.2884 - val_loss: 0.
2960 - val_mae: 0.3618
Epoch 89/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1681 - mae: 0.2862 - val_loss: 0.
2828 - val_mae: 0.3554
Epoch 90/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1645 - mae: 0.2805 - val_loss: 0.
2860 - val_mae: 0.3538
Epoch 91/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1681 - mae: 0.2842 - val_loss: 0.
2827 - val_mae: 0.3529
Epoch 92/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1650 - mae: 0.2815 - val_loss: 0.
2812 - val_mae: 0.3569
Epoch 93/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1692 - mae: 0.2889 - val_loss: 0.
2870 - val_mae: 0.3544
Epoch 94/100
```

```
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1639 - mae: 0.2818 - val_loss: 0.
2756 - val_mae: 0.3507
Epoch 95/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.1607 - mae: 0.2786 - val_loss: 0.
2766 - val_mae: 0.3544
Epoch 96/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1698 - mae: 0.2843 - val_loss: 0.
2878 - val_mae: 0.3560
Epoch 97/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1684 - mae: 0.2852 - val_loss: 0.
2876 - val_mae: 0.3515
Epoch 98/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1568 - mae: 0.2749 - val_loss: 0.
2780 - val_mae: 0.3448
Epoch 99/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1629 - mae: 0.2803 - val_loss: 0.
2892 - val_mae: 0.3563
Epoch 100/100
465/465 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 0.1640 - mae: 0.2797 - val_loss: 0.
2802 - val_mae: 0.3520
129/129 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 0.2683 - mae: 0.3433
129/129 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

Test MAE: 0.3409247100353241
RMSE: 0.5166480852156964
R2 Score: 0.7963038651373044
```





Checking parameters of history variable

In [ ]:
```
print(history.history.keys())
```

```
dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])
```

In [ ]:
```
# for batch 64
history_64 = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=64,
    verbose=0
)
```

In [ ]:
```
test_loss_64, test_acc_64 = model.evaluate(X_test, y_test, verbose=0)
```

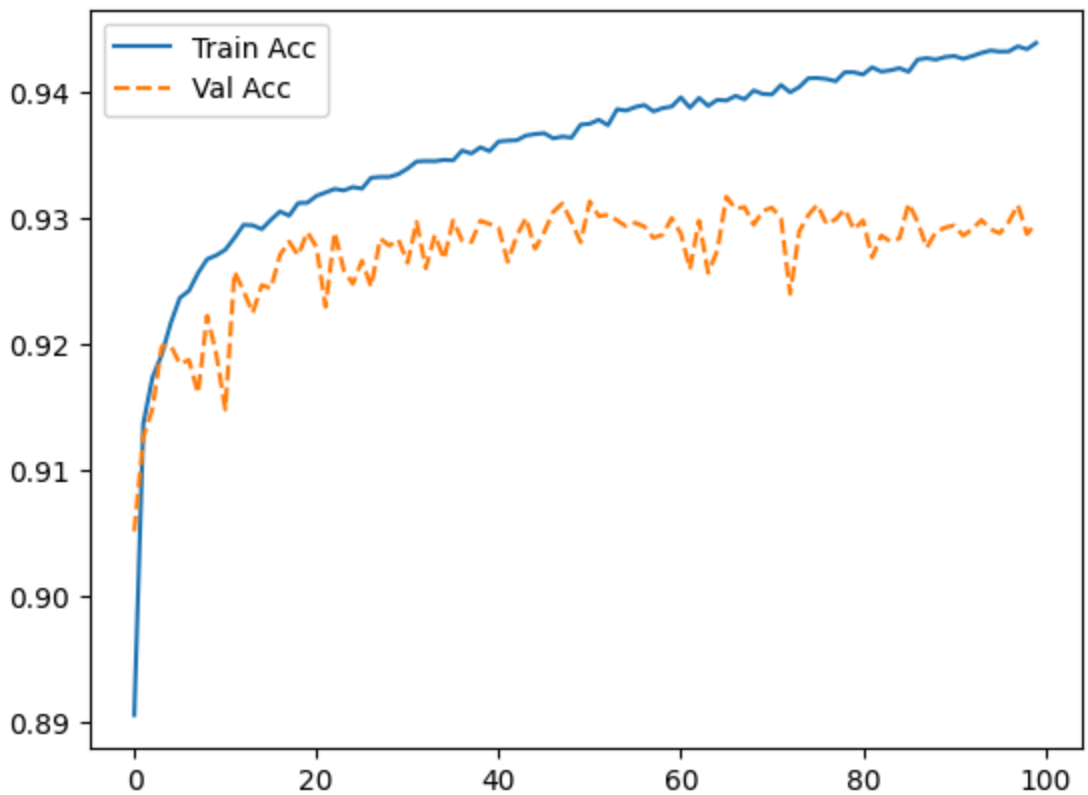
In [ ]:
```
# for batch 128
history_128 = model.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=100,
    batch_size=128,
    verbose=0
)
```

In [ ]:
```
test_loss_128, test_acc_128 = model.evaluate(X_test, y_test, verbose=0)
```

In [ ]:
```
print(history.history.keys())
```

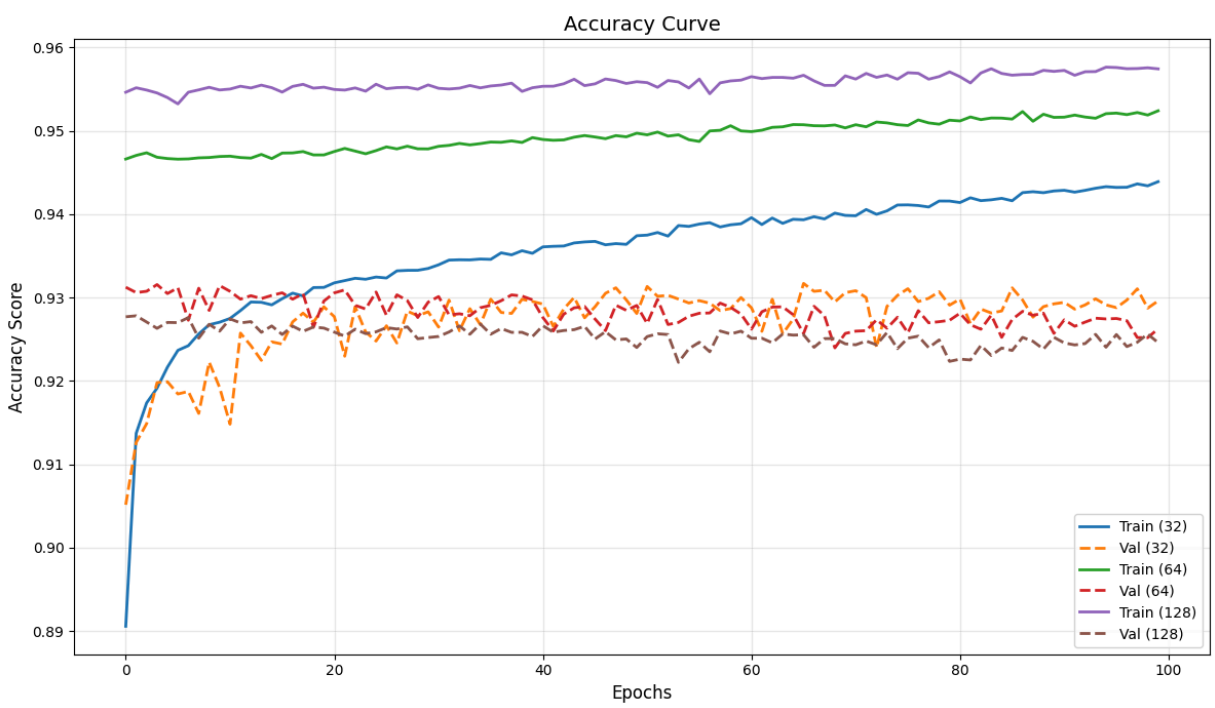
```
dict_keys(['loss', 'mae', 'val_loss', 'val_mae'])
```

In [ ]:
```
max_val = np.max(y_train)

train_acc_like = 1 - (np.array(history.history['mae']) / max_val)
val_acc_like = 1 - (np.array(history.history['val_mae']) / max_val)

plt.plot(train_acc_like, '-', label='Train Acc')
plt.plot(val_acc_like, '--', label='Val Acc')
plt.legend()
plt.show()
```



In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt

max_val = np.max(y_train)

# Convert MAE to accuracy-like
train_32 = 1 - (np.array(history.history['mae']) / max_val)
val_32   = 1 - (np.array(history.history['val_mae']) / max_val)

train_64 = 1 - (np.array(history_64.history['mae']) / max_val)
val_64   = 1 - (np.array(history_64.history['val_mae']) / max_val)

train_128 = 1 - (np.array(history_128.history['mae']) / max_val)
val_128   = 1 - (np.array(history_128.history['val_mae']) / max_val)
```

```
plt.figure(figsize=(12,7))

# Batch 32
plt.plot(train_32, '-',  linewidth=2, label='Train (32)')
plt.plot(val_32,   '--', linewidth=2, label='Val (32)')

# Batch 64
plt.plot(train_64, '-',  linewidth=2, label='Train (64)')
plt.plot(val_64,   '--', linewidth=2, label='Val (64)')

# Batch 128
plt.plot(train_128, '-',  linewidth=2, label='Train (128)')
plt.plot(val_128,   '--', linewidth=2, label='Val (128)')


plt.title("Accuracy Curve", fontsize=14)
plt.xlabel("Epochs", fontsize=12)
plt.ylabel("Accuracy Score", fontsize=12)

plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Now what happens when we change batch size 32 → 64 → 128

x_total.shape = (4128, 8)

Effect of Batch Size

- 32 -> 4128 / 32 = 129
- 64 -> 4128 / 64 = 64.5
- 128 -> 4128 / 128 = 32.25
- 256 -> 4128 / 256 = 16.125

Meaning

- In batch size 32, gradient updates occur 129 times per epoch, using 32 rows in each step.
- In batch size 64, gradient updates occur 64 times per epoch, using 64 rows in each step.
- In batch size 128, gradient updates occur 32 times per epoch, using 128 rows in each step.
- In batch size 256, gradient updates occur 16 times per epoch, using 256 rows in each step.

Observation

As the batch size increases, more rows are processed at once during each gradient update. This increases memory (RAM) usage, but reduces the number of gradient updates per epoch.

In [ ]: